Gestion des Événements et Formulaires dans React

Objectifs

1. Maîtriser la gestion des événements utilisateurs et la propagation en React.
2. Comprendre le concept des composants contrôlés pour la gestion des formulaires.
3. Implémenter la validation en temps réel des champs de saisie.
4. Gérer la soumission asynchrone de formulaires et le retour d'état.

1. Gestion des Événements

1.1 Les Événements Synthétiques

React encapsule les événements natifs du navigateur dans un objet inter-plateforme appelé **SyntheticEvent**. Cet objet possède la même interface que l'événement natif, garantissant un comportement identique sur tous les navigateurs.

Contrairement au HTML classique où les gestionnaires d'événements s'écrivent en minuscules (`onclick="..."`), React utilise la syntaxe **camelCase** (`onClick`, `onChange`, `onSubmit`).

JavaScript

```
export default function ActionButton() {
  function handleClick(event) {
    // event est un SyntheticEvent
    console.log('Type d'événement :', event.type);
    console.log('Cible :', event.target);
  }

  return (
    <button onClick={handleClick}>
      Exécuter l'action
    </button>
  );
}
```

1.2 Prévention du Comportement Par défaut

Pour empêcher le comportement natif d'un élément (comme le rechargement de page lors de la soumission d'un formulaire `<form>`), on appelle explicitement la méthode `preventDefault()` sur l'objet événement.

```javascript
function handleSubmit(event) {
  event.preventDefault(); // Bloque le rechargement de la page du navigateur
  console.log('Formulaire intercepté par React.');
}
```

---

2. Formulaires et Composants Contrôlés

2.1 Composants Non Contrôlés vs Contrôlés

En React, il existe deux approches pour gérer les éléments de formulaire (input, textarea, select) :

1. **Composants Non Contrôlés :** La valeur du champ est gérée directement par le DOM natif. On accède à sa valeur au moment de la soumission via une référence (`useRef`). Cette approche est proche du HTML traditionnel.
2. **Composants Contrôlés (Recommandé) :** La valeur de l'élément de formulaire est entièrement pilotée par l'état local du composant (`useState`). Chaque frappe clavier met à jour l'état, qui en retour redessine l'input.

2.2 Mise en Place d'un Input Contrôlé

```javascript
import { useState } from 'react';

export default function StationSearchInput() {
  const [searchTerm, setSearchTerm] = useState('');

  function handleChange(event) {
    setSearchTerm(event.target.value);
  }

  return (
    <div className="form-group">
      <label htmlFor="search">Rechercher une station :</label>
```

```
      <input
        id="search"
        type="text"
        value={searchTerm} // La valeur est dictée par l'état
        onChange={handleChange} // Chaque modification met à jour l'état
        placeholder="Nom ou code..."
      />
      <p>Recherche en cours : {searchTerm}</p>
    </div>
  );
}
```

2.3 Gestion de Formulaires Multi-Champs avec un État Objet

Plutôt de déclarer un `useState` par champ de saisie, on regroupe l'ensemble des données du formulaire dans un objet unique, mis à jour dynamiquement à l'aide de l'attribut `name` et de l'opérateur de décomposition (*Spread Operator*).

JavaScript

```
import { useState } from 'react';

export default function StationForm() {
  const [formData, setFormData] = useState({
    name: '',
    altitude: '',
    status: 'online'
  });

  function handleChange(event) {
    const { name, value } = event.target;
    setFormData(prev => ({
      ...prev,
      [name]: value // Mise à jour dynamique de la clé ciblée
    }));
  }

  function handleSubmit(event) {
    event.preventDefault();
    console.p('Données soumises :', formData);
  }

  return (
    <form onSubmit={handleSubmit}>
      <div>
```

```
        <label>Nom de la station :</label>
        <input
          type="text"
          name="name"
          value={formData.name}
          onChange={handleChange}
        />
      </div>

      <div>
        <label>Altitude (m) :</label>
        <input
          type="number"
          name="altitude"
          value={formData.altitude}
          onChange={handleChange}
        />
      </div>

      <div>
        <label>Statut :</label>
        <select name="status" value={formData.status} onChange={handleChange}>
          <option value="online">En ligne</option>
          <option value="offline">Hors ligne</option>
        </select>
      </div>

      <button type="submit">Enregistrer la station</button>
    </form>
  );
}
```

---

3. Validation et Soumission

3.1 Validation Synchrone en Temps Réel

La validation d'un formulaire consiste à vérifier la conformité des données saisies avant de les autoriser. On peut stocker les erreurs dans un état dédié (`errors`) et désactiver le bouton de soumission ou afficher des messages d'avertissement contextuels.

JavaScript

```
import { useState } from 'react';

export default function ValidatedStationForm() {
  const [name, setName] = useState('');
  const [error, setError] = useState('');

  function handleChange(e) {
    const val = e.target.value;
    setName(val);

    if (val.trim().length < 3) {
      setError('Le nom de la station doit comporter au moins 3 caractères.');
    } else {
      setError('');
    }
  }

  return (
    <div className="card">
      <input
        type="text"
        value={name}
        onChange={handleChange}
        placeholder="Nom de la station"
      />
      {error && <p className="error-text" style={{ color: 'red' }}>{error}</p>}
    </div>
  );
}
```

3.2 Gestion des États de Soumission (Chargement et Succès)

Lorsqu'un formulaire envoie des données vers une API, il est indispensable de gérer l'état de l'interface pour informer l'utilisateur :

- Désactivation des champs et du bouton pendant la requête (`isSubmitting`).
- Affichage d'un message de confirmation en cas de succès.
- Affichage d'une alerte en cas d'échec réseau.

---

4. Synthèse

1. **Événements :** React utilise des `SyntheticEvent` uniformes et normalisés en camelCase pour capter les interactions utilisateurs.

2. **Composants Contrôlés :** La valeur des champs de formulaire est liée à l'état local du composant (`useState`), garantissant un contrôle total de la donnée.
3. **Formulaires Multi-Champs :** L'utilisation d'un objet d'état combiné à l'attribut `name` simplifie grandement la gestion des formulaires complexes.
4. **Validation :** L'interception de la soumission via `preventDefault()` permet de valider les données et de gérer les états de chargement avant transmission.